

Homework 4

Diya Jain U81673988

1.

Step	Action	Multiplier	Multiplicand	Product
0	Initial values	010 001	000 000 010 110	000 000 000 000
1	1a: 1 → Product = Product + Mcand	010 001	000 000 010 110	000 000 010 110
	2: Shift left multiplicand	010 001	000 000 101 100	000 000 010 110
	3: Shift right multiplier	001 000	000 000 101 100	000 000 010 110
2	1: 0 → No operation	001 000	000 000 101 100	000 000 010 110
	2: Shift left multiplicand	001 000	000 001 011 000	000 000 010 110
	3: shift right multiplier	000 100	000 001 011 000	000 000 010 110
3	1: 0 → no operation	000 100	000 001 011 000	000 000 010 110
	2: shift left multiplicand	000 100	000 010 110 000	000 000 010 110
	3: shift right multiplier	000 010	000 010 110 000	000 000 010 110
4	1a: 0 → product = Product + Mcand	000 100	000 010 110 000	000 000 010 110
	2: shift left multiplicand	000 100	000 101 100 000	000 000 010 110
	3: shift right multiplier	000 001	000 101 100 000	000 000 010 110
5	1a; 1 → product = Product + Mcand	000 001	000 101 100 000	000 101 110 110
	2: shift left multiplicand	000 001	001 011 000 000	000 101 110 110
	3: shift right multiplier	000 000	001 011 000 000	000 101 110 110
6	1: 0 → no operation	000 000	001 011 000 000	000 101 110 110
	2: shift left multiplicand	000 000	010 110 000 000	000 101 110 110
	3: shift right multiplier	000 000	010 110 000 000	000 101 110 110

2.

Step	Action	Multiplicand	Product/ Multiplier
0	Initial values	010 110	000 000 010 001
1	1a: 1 → Product = Product + Mcand	010 110	010 110 010 001
	2: shift right product	010 110	001 011 001 000
2	1: 0 → no operation	010 110	001 011 001 000
	2: shift right multiplier	010 110	000 101 100 100
3	1: 0 → no operation	010 110	000 101 100 100
	2: shift right multiplier	010 110	000 010 110 010
4	1: 0 → no operation	010 110	000 010 110 010
	2: shift right multiplier	010 110	000 001 011 001
5	1a: 1 → product = product + Mcand	010 110	010 111 011 001
	2: shift right multiplier	010 110	001 011 101 100
6	1: 0 → no operation	010 110	001 011 101 100
	2: shift right multiplier	010 110	000 101 110 110

3.

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0100 0000	0000 1111
1	Remainder = remainder - divisor	0000	0100 0000	1100 1111
	Remainder<0, Shift left quotient	0000	0100 0000	0000 1111
	Shift divisor right	0000	0100 0000	0000 1111
2	Remainder = remainder – divisor	0000	0100 0000	1110 1111
	Remainder<0, Shift left quotient	0000	0100 0000	0000 1111

	Shift divisor right	0000	0001 0000	0000 1111
3	Remainder = remainder – divisor	0000	0001 0000	1111 1111
	Remainder<0, Shift left quotient	0000	0001 0000	0000 1111
	Shift divisor right	0000	0000 1000	0000 1111
4	Remainder = remainder – divisor	0000	0000 1000	0000 0111
	Remainder<0, Shift left quotient	0001	0000 1000	0000 0111
	Shift divisor right	0001	0000 0100	0000 0111
5	Remainder = remainder - divisor	0001	0000 0100	0000 0011
	Remainder<0, Shift left quotient	0011	0000 0100	0000 0011
	Shift divisor right	0011	0000 0010	0000 0011

4.

Code 1:

```
addi x3, x3, 1 # This instruction adds an immediate value of 1 to the value in register `x3`.
slli x3, x3, 62 #This instruction left shifts (logical shift left) the value in register `x3` by 62 bits.
addi x4, x4, 7 #This instruction adds an immediate value of 7 to the value in register `x4`.
mul x5, x4, x3 #This instruction multiplies the values in registers `x4` and `x3` and stores the result in `x5`.
mulh x6, x4, x3. #This instruction performs a 32x32-bit signed multiplication of the values in registers `x4` and `x3` and stores the high 32 bits of the result in `x6`.
```

Interpreting the register results:

1. After the first instruction, `x3` will contain its original value plus 1.
2. The second instruction left shifts `x3` by 62 bits. Depending on the initial value of `x3`, this might effectively clear most bits, leaving only the least significant bit (LSB) set to the value of the most significant bit of the original `x3`.
3. After the third instruction, `x4` will contain its original value plus 7.
4. The fourth instruction multiplies the values in `x4` and `x3` and stores the result in `x5`. The result in `x5` is the product of these two values.
5. The fifth instruction calculates the high 32 bits of the 64-bit signed multiplication result of `x4` and `x3` and stores it in `x6`. This can be used to obtain the most significant bits of a large multiplication result.

Code 2:

```
addi x3, x3, 1 #This instruction adds an immediate value of 1 to the value in register `x3`.
slli x3, x3, 63 #This instruction left-shifts (logical shift left) the value in register `x3` by 63 bits.
addi x4, x4, 1 #This instruction adds an immediate value of 1 to the value in register `x4`.
mul x5, x4, x3. #This instruction multiplies the values in registers `x4` and `x3` and stores the result in `x5`
mulhsu x6, x4, x3 #This instruction calculates the high 32 bits of the 64-bit signed multiplication result of the values in registers `x4` and `x3`.
```

Interpreting the register results:

1. After the first instruction, `x3` will contain its original value plus 1.
2. The second instruction left shifts `x3` by 63 bits. This effectively clears most bits, leaving only the least significant bit (LSB) set to the value of the most significant bit of the original `x3`. The value in `x3` will effectively become 1 if it wasn't already.

3. After the third instruction, `x4` will contain its original value plus 1.
4. The fourth instruction multiplies the values in `x4` and `x3` and stores the result in `x5`. The result in `x5` is the product of these two values.
5. The fifth instruction calculates the high 32 bits of the 64-bit signed multiplication result of `x4` and `x3`, with the high bits taken from `x3`. The result is stored in `x6`.

Code 3:

```
addi x3, x3, 1 #This instruction adds an immediate value of 1 to the value in register `x3`.
slli x3, x3, 63 #This instruction left-shifts (logical shift left) the value in register `x3` by 63 bits.
addi x4, x4, 1 #This instruction adds an immediate value of 1 to the value in register `x4`.
mul x5, x4, x3 #This instruction multiplies the values in registers `x4` and `x3` and stores the result in `x5`.
mulhsu x6, x3, x4 #This instruction calculates the high 32 bits of the 64-bit signed multiplication result of the
values in registers `x3` and `x4`.
```

Interpreting the register results:

1. After the first instruction, `x3` will contain its original value plus 1.
2. The second instruction left-shifts `x3` by 63 bits. This effectively clears most bits, leaving only the least significant bit (LSB) set to the value of the most significant bit of the original `x3`. The value in `x3` will effectively become 1 if it wasn't already.
3. After the third instruction, `x4` will contain its original value plus 1.
4. The fourth instruction multiplies the values in `x4` and `x3` and stores the result in `x5`. The result in `x5` is the product of these two values.
5. The fifth instruction calculates the high 32 bits of the 64-bit signed multiplication result of `x3` and `x4`, with the high bits taken from the signed value in `x3`. The result is stored in `x6`.

5.

Code 1:

```
addi x7, x7, 45 #This instruction adds an immediate value of 45 to the value in register `x7`.
addi x4, x4, 3 #This instruction adds an immediate value of 3 to the value in register `x4`.
div x5, x7, x4 #This instruction divides the value in register `x7` by the value in register `x4` and stores the
quotient in `x5`.
div x6, x4, x7 #This instruction divides the value in register `x4` by the value in register `x7` and stores the
quotient in `x6`.
```

Interpreting the register results:

1. After the first instruction, `x7` will contain its original value plus 45.
2. After the second instruction, `x4` will contain its original value plus 3.
3. The third instruction performs the division of `x7` by `x4` and stores the quotient in `x5`. The result in `x5` is the integer quotient of the division.
4. The fourth instruction performs the division of `x4` by `x7` and stores the quotient in `x6`.

Code 2:

```
addi x7, x7, -46 #This instruction adds an immediate value of -46 to the value in register `x7` and stores the
result back in `x7`. The negative immediate value effectively subtracts 46 from the original value in `x7`.
```

`addi x4, x4, 3` #This instruction adds an immediate value of 3 to the value in register `x4`.
`div x5, x7, x4` #This instruction divides the value in register `x7` by the value in register `x4`.
`rem x8, x7, x4` #This instruction computes the remainder when the value in register `x7` is divided by the value in register `x4`, and it stores the remainder in `x8`.

Interpreting the register results:

1. After the first instruction, `x7` will contain its original value minus 46.
2. After the second instruction, `x4` will contain its original value plus 3.
3. The third instruction performs the division of `x7` by `x4` and stores the quotient in `x5`. The result in `x5` is the integer quotient of the division.
4. The fourth instruction computes the remainder when `x7` is divided by `x4` and stores the remainder in `x8`. The remainder is the value that, when multiplied by `x4` and added to the quotient in `x5`, results in the original value in `x7`.

Code 3:

`addi x7, x7, -46` #This instruction adds an immediate value of -46 to the value in register `x7` and stores the result back in `x7`. The negative immediate value effectively subtracts 46 from the original value in `x7`.
`addi x4, x4, 3` #This instruction adds an immediate value of 3 to the value in register `x4` and stores the result back in `x4`.
`divu x5, x7, x4` #This instruction performs an unsigned division, dividing the value in register `x7` by the value in register `x4` and stores the quotient in `x5`.

Interpreting the register results:

1. After the first instruction, `x7` will contain its original value minus 46.
2. After the second instruction, `x4` will contain its original value plus 3.
3. The third instruction performs an unsigned division of `x7` by `x4` and stores the quotient in `x5`.
Unsigned division treats the operands as non-negative integers and does not consider the sign bit.